

Finding Deep-Hidden Bugs in Android Apps via Functional Semantics Guided Exploration

Siyi Gu[1], Xiaoqiang Liu[1(✉)], Hui Guo[1], Bochun Cao[1], Baiyan Li[1], Lizhi Cai[2], and Hu Yun[2]

[1] School of Computer Science and Technology, Donghua University, Shanghai, China
liuxq@dhu.edu.cn

[2] Shanghai Key Laboratory of Computer Software Evaluating and Testing, Shanghai, China

Abstract. As Android apps get more complicated, current automated testing techniques are not sufficient for discovering deep-hidden bugs. The deep-hidden bugs are triggered by sequential Data Manipulation Functions(DMFs), which often represents functional semantics (Create, Read, Update, Delete and Search). The Data Manipulation Function Sequence (DMFSQ) is composed of DMFs. Testing by DMFSQs can closely mirror real user interactions, and provide a more in-depth evaluation of the app's functions. Therefore, to find the deep-hidden bugs, we propose a novel functional semantics guided exploration strategy based on DMFs and DMFSQs. Our idea is that we can (1) extract DMFs from the apps, and (2) use functional semantics of DMFs in the execution trace to guide the test exploration. To achieve this idea, we conducted a study on the GUI of various apps' DMFs to identify accurate and universal GUI patterns as the extraction rule. With the rule, an algorithm is developed to extract DMFs from apps while detecting regular bugs during depth-first test exploration. Based on extracted DMFs, we design a exploration strategy based on Q-learning, a classic reinforcement learning algorithm for detecting deep-hidden bugs. We have implemented our idea as an automated testing tool DmsDroid. DmsDroid can not only discover regular bugs, but also generate meaningful DMFSQs to detect deep-hidden bugs. The evaluation results of DmsDroid on 16 real-world apps shows that DmsDroid outperforms the state-of-practice Android testing techniques in terms of code coverage and bug detection. So far, 7 of our reported bugs have been confirmed.

Keywords: deep-hidden bugs · data manipulations function · Android app testing · reinforcement learning

1 Introduction

As the number of Android apps continue to increase, the complexity of these apps also rises [38]. Consequently, discovering deep-hidden bugs has become increasingly challenging. Specifically, even if performing all single executable

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024
W.-N. Chin and Z. Xu (Eds.): TASE 2024, LNCS 14777, pp. 146–164, 2024.
https://doi.org/10.1007/978-3-031-64626-3_9

UI events in the app also cannot completely guarantee that this app is bug-free. One obvious phenomenon is that the app can successfully execute a single function, but the diverse combinations of multiple functions could crash it. Thus, a mature testing tool should not only test each individual function, but also needs to consider the relationship between functions to find deep-hidden bugs in the series of operations.

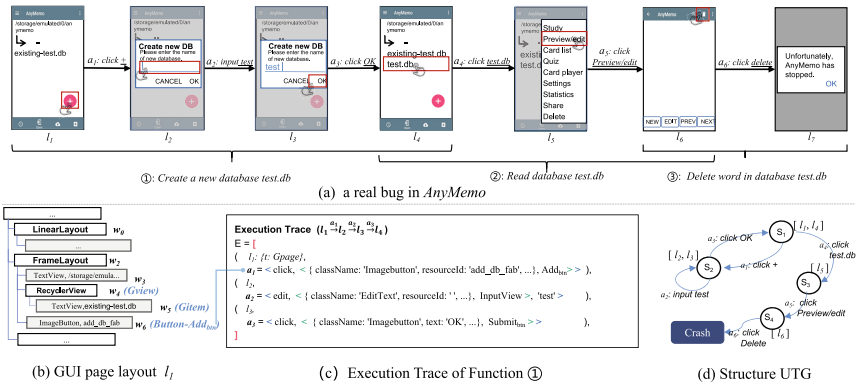


Fig. 1. *AnyMemo.* (a) is the scenario that a real bug triggered by continuous function execution. (b) shows the GUI page layouts l_1 and l_4. (c) is the execution tarce of function ①. (d) is the structure UTG of (a).

An Illustrative Example. Figure 1.a shows a real bug triggered by the combination of multiple functions in AnyMemo. AnyMemo is a vocabulary memorization app with over 100K downloads on Google Play. The user executes 3 functions continuously. ① Creates a database test.db (see $l_1 \sim l_4$). The user clicks the button +, enters the text test and clicks the button OK. ② Read the database test.db (see $l_4 \sim l_6$). The user selects this database test.db in the file list and clicks Preview/edit operation to view the vocabularies in test.db. ③ Delete the vocabularies in test.db (see $l_6 \sim l_7$). The user clicks the button delete to remove the vocabularies. However, because the newly created test.db is an empty database and there is no vocabulary, the app crashes when performing the function ③.

This bug is triggered when three individual data manipulation functions (DMFs) are executed sequentially. We call such an execution sequence (e.g., ①→②→③) as data manipulation functions sequence(DMFSQ). A DMFSQ contains a series of functional semantics (Create, View, Update, Delete, Search), closely mirroring the actions of real users. From the example, we infer that deep-hidden bugs often lurk within DMFSQs. By automating the generation and execution of DMFSQs, the hidden bugs can be detected in the overall functional flow of the app. However, detecting such deep-hidden bugs by DMFSQs is non-trivial due to two major reasons. First, the User Interface (UI) characteristic of apps is diverse, it is difficult to identify different kinds of DMFs for each app

under test (AUT). Second, the combinations of DMFs are varied, which meaning that the functional semantics contained in DMFSQs are complex and diverse. For example, triggering the bug in Fig. 1.a required a specific execution order of DMFSQ of ①→②→③.

Limitations of Existing Testing Techniques. Many techniques have been employed for automated testing in Android apps. However, existing techniques have two major limitations in detecting these bugs triggered by combinations of DMFs. First, the majority of existing techniques [6,9,11,23,24] are designed to respond to single GUI operation or individual function. Thus, these techniques struggle to generate meaningful test cases containing diverse combinations of multiple DMFs. Second, some testing techniques [29,36] can extract short sequences of events and combine them in a way of cross-enumeration to find bugs. However, owing to the lack of prior knowledge before testing, they may lead to an exponential increase in testing workload as the number of function points grows. Therefore, these techniques may not be so efficient at testing the bugs triggered by function combinations.

Our Approach. To this end, we propose a functional semantics guided exploration strategy to efficiently find these deep-hidden bugs. Our idea is that, we should (1) extract the DMFs automatically during the first testing exploration, and (2) further utilize these DMFs to effectively guide the testing exploration toward finding the deep-hidden bugs. Inspired by reinforcement learning-based techniques [4,30,31,35,37], we design a functional semantics guided exploration strategy based on Q-learning. The reward function in the strategy considers the current operation's relevance to DMFs, the variety of explored DMFs, and the functional semantics of explored DMFSQs during the exploration. Based on this strategy, the Q-learning agent can explore more meaningful DMFSQs, which contains different DMFs in effective orders.

At the same time, in order to find the DMFs in various kinds of app as completely as possible, the co-authors conducted an Empirical Study, summarizing patterns in the DMF GUI of 81 apps. We identify universal GUI change patterns of DMFs as extraction rules. This study serves as the foundation for the real-time automated extraction algorithm of DMFs. The extraction algorithm extracts DMFs simultaneously with depth-first test exploration, which can detect regular bugs. With the extracted DMFs, we further explore the app via functional semantics guided strategy to detect the bugs triggered by DMFSQs.

Contributions. Overall, our contributions can be summarized as follows:

- We surveyed the GUI of 81 apps, formulated a set of DMF extraction rules.
- We designed a real-time DMF extraction algorithm to extract DMFs simultaneously while finding regular bugs with depth-first exploration.
- We proposed a functional semantics guided exploration strategy based on Q-learning to effectively explore more DMFSQs to find deep-hidden bugs.

Our approach DmsDroid and the data used in the experiments can be found on GitHub[1]. The evaluation on 16 real-world apps shows DmsDroid can discover more bugs in some complex cases.

[1] https://github.com/TestingDMS/DMSDroid.

2 Background

2.1 DMFs in Apps

Data manipulation functionalities (DMFs) perform the common CRUDS operations (create, read, update, delete, search) to handle app-specific data [33]. Users generally perform Data Manipulation Functions (DMFs) through the graphical user interface (GUI) in apps. GUI [16] is a fundamental component of Android apps, providing an easy-to-use and intuitive interface that enables users to communicate with the app. The GUI typically consists of a set of page layouts (see $l_1 \sim l_6$ in Fig. 1.a), and represent the actions that users can perform on that interface (see $a_1 \sim a_6$ in Fig. 1.a). The layout-action pair (l, a) can be combined to form an execution trace $[(l_1, a_1), ...(l_i, a_i), ...(l_n, a_n)]$. In Fig. 1.a, three layout-action pairs are executed continuously $[(l_1, a_1), (l_2, a_2), (l_3, a_3)]$ and achieve the functional semantics *Create a new database*. Therefore, ① is a DMF, similarly, ② and ③ are as well. The functions ①→②→③ formed a DMFSQ.

2.2 Q-Learning for Android Testing

Q-learning [37] is a widely-used model-free reinforcement learning (RL) algorithm. At time t, the agent first observes the environment to obtain the current state ($s_t \in S$) and an immediate reward ($r_t \in R$). The agent then selects an action ($a_t \in A$) based on these variables. This action causes a transition in the environment from the current state (s_t) to the next state ($s_{t+1} \in S$) and generates the subsequent reward ($r_{t+1} \in R$). The process repeats for each interaction until a specified time limit is reached. Equation (1) shows the Bellman equation for updating the action value function. where α is the learning rate, γ is the discount factor, and $max_a Q(s_{t+1}, a)$ is the maximum expectation of the next state s_{t+1} award.

$$Q(s_t, a_t) = Q(s_t, a_t) + \alpha(r + \gamma max_a Q(s_{t+1}, a) - Q(s_t, a_t)) \tag{1}$$

The Q-values of future state-action pairs, $Q(s_{t+1}, a)$, influence previous pairs $Q(s_t, a)$. With ample exploration, the Q-table values converge to their true values. Therefore, the Q-table can accurately guide the agent to maximize cumulative rewards by selecting the highest-valued actions. There are also many testing tools [15,20,30] that choose to conduct testing exploration based on Q-learning. Inspired by this, we choose Q-learning algorithm to guide the test exploration with functional semantic guidance.

3 Empirical Study

In order to summarize general and accurate GUI pattern, we did an empirical study on DMF's GUI in this section. From a batch of different apps, we describe how to analyze the GUI of the app and elaborate the specific process of defining DMF extraction rules.

3.1 App Collection

To conduct a comprehensive analysis and research on the GUI of DMFs, we analyzed the GUI of DMFs using open-source apps from GitHub and F-Droid, and popular apps from Google Play. To ensure rule universality, we studied 10 app categories, collecting 81 apps in total: 61 open-source and 20 closed-source. Figure 2 shows the statistics of these apps. The information includes the number of stars on GitHub, the number of commits on GitHub, the app rating and the number of downloads on Google Play Store, and the categories of these apps. These data shows that our study population is (1) well-maintained (containing an average of thousands of commits), (2) popular (90% of downloads exceed 1000K and ratings exceed 4.0), and (3) diverse (covering 10 categories).

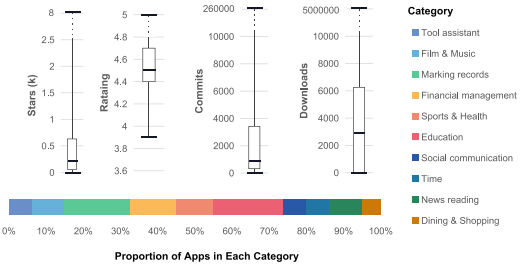


Fig. 2. Statistics of Apps in Each Category

3.2 Identify GUI Patterns of DMFs

Actions represent the user's interactions with the widgets, which can be clicking buttons on the screen, entering text, etc. The action $a = <t, w, d>$ is a tuple, where $a.t$ denotes the action's type (e.g., click, edit), $a.w$ denotes the current widget w, and $a.d$ denotes the optional data of a (e.g., the text inputted by edit).

Layouts define the arrangement and structure of the widgets within the GUI. The layout $l = <t, W>$ is a tuple. $l.W$ is the set of widgets on layout l. If there is a widget $w.c$=`Gview`, we infer that $l.t$=`Gpage`. Figure 1.b shows the details of l_1. l_1 contains widgets $w_0 \sim w_6$. The category of w_4 is `Gview`, so l_1 can be expressed as $l_1 = <$`Gpage`, $\{w_0..., w_6\}>$.

Execution trace E consists of consecutively executed layout-action pairs, i.e., E=$[(l_1, a_1), ...(l_i, a_i), ...(l_n, a_n)]$. The page layout after perform E is also the key for us to judge whether it is a complete DMF. Therefore, we define l_{end} to denote the ending page layout of execution trace E. If a GUI view with a certain data d appears in l_{end}, it represents as $d \rightarrowtail l_{end}$. Since DMF is an execution trace with specific functional semantics, we define a DMF as a 3-tuple $\sigma = <t, E, d>$, where $\sigma.t$ denotes the operation type (*Create*, *Read*, *Update*, *Delete* or *Search*). $\sigma.E$ denotes the execution trace of σ, and $\sigma.d$ denotes the optional data of σ. The DMF collection that stores σ as $\Sigma = \{\sigma_1, ..., \sigma_i, ..., \sigma_n\}$. When E satisfies DMF extraction rule, E will be extracted as a DMF σ and stored in Σ.

Based on the formal definition of various GUI elements, we spend a total of 20 person-days investigating the GUI patterns of DMFs in all 81 apps.

3.3 Rule Definition for Extracting DMFs

The DMF extraction rule is designed to identify DMFs by analyzing the GUI of each (l, a) pair and evaluating l_{end} to ascertain the data presence or absence (we refer to PbfDroid to formulate post-conditions). Based on the GUI pattern, we design the DMF extraction rule, which is shown in Table 1.

DMF extraction rule R is structured as a dictionary $<key,\ value>$. Key represents the semantics of DMF. *Value* include the GUI requirement of (l, a) and the post-condition l_{end}. In a specific DMF, there is a set of key layout-action pairs. The GUI-pattern of each key layout-action pair (l, a) can be expressed as *R.key.step*. *R.key.end* denotes the GUI requirement of l_{end}. For instance, R.create.1 denotes the first key (l, a) GUI requirement, i.e., $l.t = Gview$, $a.t = click$, $w.c = add_btn$ in the Create DMF. *R.create.end* is $d \rightarrowtail l_{end}$. Note that the post-condition of *Search* is empty due to the lack of clear restrictions on input characters. For a given search keyword, it may either be found or not found.

Table 1. Semantic extraction rule R

DMF Type	(l, a) **in** E	l_{end}
Create	① $l.t{=}Gview, a.t{=}click, w.t{=}add_{btn}$	$d \rightarrowtail End(E)$
	② $a.t{=}edit, a.w.t{=}inputview;$ $d{=}a.d$	
	③ $a.t{=}click, a.w.t{=}submit_{btn}$	
Read	① $a.t{=}click, a.w.t{=}Gitem$ $d{=}w.v.text$ or $w.v.description$	$d \rightarrowtail End(E)$
Delete	① $a.t{=}click, w.t{=}delete_{btn}$	$\neg d \rightarrowtail End(E)$
	② $a.t{=}click, a.w.t{=}submit_{btn}$	
Update	① $a.t{=}click, a.w.t{=}modify_{btn}$	$(\neg d \rightarrowtail End(E))$ $\wedge$ $(d' \rightarrowtail End(E))$
	② $a.t{=}edit, a.w.t{=}inputview;$ $d{=}a.w.v.text, d'{=}a.d$	
	③ $a.t{=}click, a.w.t{=}submit_{btn}$	
Search	① $a.t{=}click, a.w.t{=}search_{btn}$	-
	② $l.t{=}Gview, a.t{=}edit, a.w.t{=}inputview;$ $d{=}a.d$	

In the Empirical Study, we establish the theoretical basis for the GUI extraction process for DMF. By collecting apps, defining GUI elements with formal languages, and summarizing the GUI change rules of DMFs, the tool is able to deeply understand the association patterns of GUI elements in DMF.

4 Approach

This section presents the technical details of DmsDroid. Figure 3 depicts the overview of our two-stage automated testing approach. (1) $Stage_1$: Automatically extract DMFs according to semantic extraction rules during depth-first test exploration, for detecting regular bugs; (2) $Stage_2$: Explore AUT with functional semantic guidance and generate DMFSQs as meaningful test cases by the extracted DMFs, for detecting deep-hidden bugs.

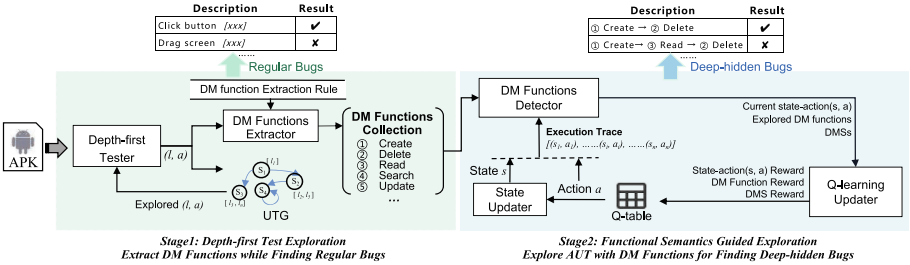


Fig. 3. Workflow of DmsDroid

4.1 DMF Extraction While Depth-First Exploration

Depth-first exploration is a systematic test exploration method used by many tools [9,13,22,36]. Through depth-first exploration, the regular problems in the apps can be discovered. Figure 1.d shows the structure UI Transition Graph (UTG) of the execution trace in Fig. 1.a. During depth-first exploration, DmsDroid generates the structure UTG of the app to avoid operations from being executed repeatedly. At the same time, the paths in the app can be fully traversed, which makes DMF fully extracted. We record and compare pairs of page layout and action (l, a) in real-time while the depth-first test exploration. Based on this idea, we design the real-time DMF extraction algorithm (Algorithm 1).

Before all, we initialize E, *step*, *type*, and data d (line 1). In dynamic exploration, we record executed operation in G (line 3). We analyze the current page's layout and actions and select an action from the actions (line 5, 6). If the layout-action pair has not been executed before, we execute this layout-action pair and continue exploration via recursive depth-first calls (line 7–11). The real-time extraction function *FunctionEventExtraction* receives the layout-action pair (l_t, a_t) as input (line 14). It first checks if the exploration is targeting a specific DMF. If (l, a) matches the initial requirement for a function type in the rule, it sets *type*, *step*, and records (l, a) in E (lines 15–18). In second condition, it verifies if the execution trace is complete and satisfies the criteria (line 19). If so, it creates a tuple σ with E, *type*, and data d, storing it in Σ, and then resets E, *type*, and *step* (lines 20, 21). In the third condition, during trace exploration but before reaching the end, if l doesn't cause a loop in E, it infers the target requirement $rule_{target}$ (line 22). If (l, a) satisfies $rule_{target}$, incrementing *step* by 1 (lines 23–26). Data d is also obtained at specific steps according to functional extraction rules R (lines 27,28).

Example 1. In Fig. 1.a, the user creates a new database *test* (①). First, the user clicks the + button (a_2) on page l_1, where $l_1.t = Gview$, $a_1.t = click$, and $a_1.w.c = add_{btn}$, satisfying the first step of DMF *Create*. We change *step* as $step = 1$, and record the execution trace as $E = [(l_1, a_2)]$. The subsequent layout-action pair needs to meet the requirements of $R.create.2$, i.e., $a.t = edit, w.c = inputview$. Fortunately, the second layout-action pair (l_2, a_2)

Algorithm 1: Real-time function extraction algorithm

```
1  E ← List(), step ← 0, type ← ⊥, d ← ⊥ // initialize execution trace, step, type, data
2  Function DepthFirstExploration(app)
3      G ← ∅ // save executed operations
4      if ¬isTimeOver() then
5          l_t, actions_t ← getPageDetails()
6          a_t ← selectRandomNotExecutedAction(actions_t, G)
7          if (l_t,a_t) ∉ G then
8              execute(l_t, a_t)
9              FunctionEventExtraction(l_t, a_t)
10             DepthFirstExploration(app)
11             G ← G ∪ (l_t, a_t)
12         SwitchBackTo(l_t)
13     return G
14 Function FunctionEventExtraction(l, a)
15     if curStep = 0 then
16         if satisfyGUIPatternInRule((l, a), (any(R.type_s.0) ∈ R)) then
17             type ← type_s, step ← 1
18             E.append(l, a)
19     else if isFinalStep(step, type) ∧ satisfyEnd(d, l_end, R.type.end) then
20         σ ← (type, E, d), Σ.append(σ)
21         Init(E, type, step, d)
22     else if notLoop(seq, s) then
23         rule_target ← getTargetStepRule(R, type, step)
24         E.append(l, a)
25         if satisfyGUIPatternInRule((l, a), rule_target) then
26             step ← step + 1
27             if shouldGetData(rule_target) then
28                 d ← GetData(rule_target)
```

satisfies the second step *R.create*.2, i.e., $step = 2$, $E = [(l_1, a_1), (l_2, a_2)]$. Meanwhile, according to the rule, we obtain the data $d = test$. The next layout-action pair (l_3, a_3) satisfies *R.create*.3, leading us to increase *step* and record execution trace $E = [(l_1, a_1), (l_2, a_2), (l_3, a_3)]$. As $step = 3$, which is the final step, we check $d \rightarrowtail l_{end}$ to confirm that the execution trace complies with the DMF *Create*, and then set $\sigma = [t, E, d]$, storing it in Σ.

4.2 Functional Semantics Guided Q-Leaning Exploration

Reinforcement learning provides a method for systematically exploring app functions [4,19,30,35]. In order to discover more deep-hidden bugs, it is necessary to consider the functional semantics to guide subsequent exploration. DmsDroid uses Q-learning to explore the app guided by the functional semantics.

Formulating Android Testing as MDP. An Android GUI testing problem is mathematically formalized as an MDP, <*S,A,P,R*>. The definition of S, A, and R in Android GUI testing significantly impacts the effectiveness and efficiency of the testing process. We do not definite P, because Q-learning does not assume to know the transition probability function [34]. We employ the following formulations:

S: States. S represents the set of all possible Android GUI states during exploration. We refer to the abstraction of state in Q-TESTING [30], and use the C-Lv4 standard in GUICC [10] to define the state of each layout.

A: Actions. A represents the set of all possible actions of the certain state. Similar to previous work [4,30,35], DMSDROID infers executable actions in the current state by analyzing the dumped widget hierarchy.

$$selectAction(s) = \begin{cases} argmax_a Q(s,a) & 1-\epsilon \\ random\ UI\ event & \frac{1}{2}\epsilon \\ random\ system\ event & \frac{1}{2}\epsilon \end{cases} \tag{2}$$

Equation (2) describes the ϵ -*greedy* policy that is utilized by DMSDROID to select the next action. The ϵ value is set to 0.2 in DMSDROID. DMSDROID selects the event with the highest Q-value in the Q-table with a probability of 0.8. For random UI events and system-level events, it has a probability of 0.1 to select.

R: Reward. The reward in GUI testing is a numerical value assigned to a specific state or state-action pair. To explore the apps more effectively, we take the exploration of DMF and DMFSQs as important considerations.

Reward Function. DMSDROID aims to effectively guide the test agent in exploring DMFs and discovering deep-hidden bugs. The functional reward r_{fun} is defined by (3). Based on the DMFs extracted in the depth-first testing exploration, we calculate the functional semantics reward from three aspects. First, we use $h(s,a)$ to calculate whether the current operation matches any key step of the function extraction rule. If it matchs, rewards 1, otherwise rewards 0. Then we use the sliding window algorithm to obtain the part of the DMFs E_{fun} in the entire execution trace E. We calculate the proportion of explored DMFs in the execution trace, i.e., $\frac{|E_{fun}|}{|E|}$. Finally, we set a semantic sequence Seq_{bug} to calculate the proportion of DMFSQs that are valid and promising to trigger bugs, which is recorded as $\frac{g(E_{fun},Seq_{bug})}{|E_{fun}|}$.

$$r_{fun} = h(s,a) + \frac{|E_{fun}|}{|E|} + \frac{g(E_{fun}, Seq_{bug})}{|E_{fun}|} \tag{3}$$

Furthermore, solely depending on rewards linked to functional semantics could result in explorers getting stuck in exploration errors characterized by sparse rewards. Hence, drawing from prior research, we introduced the r_{basic} (4) parameter to mitigate this risk. $f(s,a,E)$ is used to calculate the frequency of occurrence of (s,a) in the current execution trace E.

$$r_{basic} = -f(s,a,E) \tag{4}$$

The final reward function can be expressed as $r_{fun} + r_{basic}$.

Functional Semantics Guided Exploration. We employ Q-learning algorithm to achieve a functional semantics guided exploration strategy. The function *FunctionalGuidedExploration* in Algorithm 2 describes the detail of functional

Algorithm 2: Functional Semantics Guided Exploration

```
1  Function FunctionGuidedExploration(Σ, app)
2      Q ← InitQTable(Σ) // initialize Q-table
3      C ← ∅ // save executed DMFs
4      E ← List() // save the execution trace
5      while ¬isTimeOver() do
6          s_t, actions_t ← getPageDetails()
7          foreach action_t ∈ actions_t do
8              if action_t ∉ Q then
9                  setQvalue(Q, s_t, action_t, Q_init)
10         a_t ← selectAction(Q, s_t)
11         s_{t+1} ← executeAction(app, a_t)
12         E.append(s_t, a_t)
13         E_fun ← getCurFn(seq) // Get current DMF using a sliding window
14         r_fun, r_basic ← getReward(s_t, a_t, E, E_fun, C)
15         Q(s_t, a_t) ← Q(s_t, a_t) + α(r_fun + r_basic + γ max_a Q(s_{t+1}, a) − Q(s_t, a_t))
16         if E_fun ≠ null ∧ intent ∉ C then // new DMF
17             C ← C ∪ E_fun
18     return Q
```

semantics guided exploration strategy. First, DmsDroid assign an inital value of the Q-table based on the extracted DMFs' collection Σ (line 2). DmsDroid get the detail of GUI page, including state s_t and find out all the executable actions $actions_t$ (line 5). Q-table records the value of each action or assign an initial value if it is executed for the first time (line 8, 9). Then DmsDroid employs (2) to select an action a_t. DmsDroid executes a_t and saves the exploring state and action into E (line 10–12). Then DmsDroid obtain the part of the DMF E_{fun} in the entire execution trace E (line 13). Based on s_t, a_t and E_{fun}, DmsDroid calculates the functional reward r_{fun} by (3) and the basic reward by (4) (line 14, 15). After extensive experimentation, γ is set to 0.9 in DmsDroid to strengthen the impact of unexplored functions. The $Q_{max}(s_{t+1}, a_{t+1})$ will be relatively higher for unexplored DMFs. Benefiting from the propagation property of Q-value, the change of reward can guide the agent in DmsDroid to explore more meaningful test cases (line 16). For subsequent guided exploration, when the agent discovers new DMFs, it store them in the set C (line 16–17).

DmsDroid extracts valuable insights by calculating the explored DMFs and potential error-triggering DMFSQs. The Q-learning agent can explore different cases that contain multiple DMFs and various execution orders of DMFs. Additionally, it employs a frequency penalty to optimize action exploration, thereby enhancing learning and system stability with each iteration.

5 Evaluation

In our experimental evaluation, we seek to answer three questions:

RQ1: **Functions Extraction.** How efficiently does DmsDroid extract DMFs from the apps, as measured by concrete metrics?

RQ2: **Testing Efficiency.** How does the testing coverage and bug detection performance of DmsDroid compare to other popular testing tools?
RQ3: **DMFs Usefulness.** How effective are the DMFs and DMFSQs in improving the testing effectiveness?

5.1 Experiment Setup

Tool Implementation. DmsDroid reuses and extends the testing techniques: DroidBot [22] is the foundation for our tool for generating UTG. Android Debug Bridge(ADB) [2] for sending UI events, and Logcat [3] for recording the execution of DMFs and monitoring runtime exceptions.

Table 2. Testing Results of 16 open-source apps

Target APP		% Coverage							# Crashes						
App Name	ELOC	M	Q	Cb	Pb	Dm	Dm^{α}	Dm^{β}	M	Q	Cb	Pb	Dm	Dm^{α}	Dm^{β}
BookCatalogue	75241	12	38	42	32	36	32	34	1	2	2	3	1	2	3
Amaze	60325	27	34	40	30	40	23	28	1	4	3	0	3	3	2
K9Mail	93455	18	15	26	21	27	22	25	1	1	1	0	2	2	1
AnyMemo	30661	31	55	64	45	48	55	35	1	0	1	1	3	2	1
BudgetWatch	9863	43	45	63	40	57	59	46	1	1	0	2	1	1	2
CatimaLoyalty	11547	34	22	61	48	66	50	55	1	0	2	1	1	0	0
Uhabits	10296	35	40	38	45	42	30	42	2	0	0	1	1	3	2
Runnerup	40001	28	24	38	36	35	30	44	0	2	0	0	0	0	0
TrackandGraph	16991	19	43	18	40	45	47	52	0	0	1	0	2	0	2
PhonographPlus	43261	14	20	14	26	30	34	44	0	0	0	1	2	1	1
GrowTracker	15339	22	32	45	31	50	34	36	1	1	1	1	2	1	2
Notally	1559	37	45	56	36	54	60	58	0	0	1	0	1	0	0
OmniNotes	28462	22	21	29	30	44	37	23	2	0	1	0	2	2	1
Activitydiary	9254	39	71	55	54	67	53	73	0	1	0	2	0	0	0
Markor	25674	21	-	-	46	45	43	37	1	-	-	2	1	1	1
Antennapod	117753	15	28	19	23	30	24	21	0	0	0	1	0	0	1
Average/Sum	-	26.06	35.53	40.53	36.43	44.75	39.56	40.81	12	12	13	15	22	18	19

[1] Column **M**, **Q**, **Cb**, **Pb** and **Dm** denote Monkey, Q-testing, ComboDroid and DmsDroid.
[2] Markor failed during testing with Q-testing and ComboDroid, resulting in empty data.

App Subjects. The first column of Table 2 lists the 16 evaluation subjects, selected according to the following rules. First, we selected the five large apps evaluated in existing work (with ELOC greater than 50,000): BookCatelogue, AmazeFileManager, AntennaPod, Markor and K9-mail. Second, we selected six apps with at least 5K Downloads evaluated in the existing work: Budget Watch, Runnerup, AnyMemo, Uhabits, OmniNotes and Catima. Additionally, we randomly selected three popular open-source apps from GitHub (at least 100 stars by 2023): ActivityDiary, PhonographPlus, Track&Graph, and GrowTracker.

Execution Environment. All the 16 open-source apps ran on Android emulators (Marshmallow version, Android 7.0) deployed on a 64-bit Ubuntu physical machine with Intel(R) Core(TM) i5-12490F CPU and 32 GB RAM. For each emulator, we allocated 2 GB RAM and stored different types of external files in SDCard for ensuring file access of apps.

Bug and Coverage Metrics. We measure the thoroughness of testing using two metrics. The first is bytecode instruction coverage collected by JaCoCo [17], as a higher code coverage strongly correlates to a better exercise of an app's functions. Second, we study whether the techniques can manifest previously bugs by examining the Android system's logs. For bugs, we refer to the definition of fault in stoat [32]. Bugs are identified by crash or exception lines in stack traces (exceptions without AUT package name keywords are excluded). We review the bug log, remove duplicates, exclude non-app errors (e.g., network errors).

Evaluation Setup of RQ1. To evaluate the extraction ability of DMFs in each app, we manually organized all relevant DMFs in the AUTs. The co-authors analyze the GUIs of the DMFs in each app and collect them as a basis for evaluation. To ensure the completeness and accuracy, each author first collects the DMFs in AUTs and then discussed, adjusted and merged all the DMFs collected by each individual. In total, we collected 133 DMFs for 16 apps.The functions which are collected manually are regarded as correctly extracted.

We employ three evaluation metrics to evaluate the performance of DMFs extraction. Precision is the proportion of DMFs that are correctly extracted among all extracted functions. Recall is the proportion of DMFs that are correctly extracted among all DMFs. F1-score is the harmonic mean of precision and recall, which combine both of the two metrics above. For all metrics, a higher value represents better performance. In order to prevent the inaccuracy, we extracted DMFs 5 times for 16 apps, and each extraction took 1 h.

Evaluation Setup of RQ2. In this part of evaluation, DmsDroid is compared with Monkey , Q-testing, PbfDroid and ComboDroid.

Monkey is a widely-used state-of-practice testing tool based on random events. Q-testing is a state-of-art testing tool that bases on Q-learning. Q-testing employs a curiosity-driven exploration strategy to select events. ComboDroid is a model-based tool employing the idea of combining short sequences to generate meaningful and extensive test cases. PbfDroid is a property-based tool with a focus on combining DMFs to detect bugs in the apps. It collects DMFs manually and explores pages related to DMFs.

According to previous work, the testing time of each tool is set to two hours for full exploration, and each tool tested 5 times. We set 200 milliseconds event interval for Monkey. Q-testing is a closed-source testing tool, so we adopted the default configuration. For ComboDroid, the 2-h testing time includes the time obtaining use cases and enmuerating use cases combos. PbfDroid is a semi-automated functional bug testing tool. The authors manually collected DMFs in advance according to the user manual and set PbfDroid to execute events with a maximum time of 2 h. We allocated 1 h for DmsDroid to extract the DMFs and 1 h for functional semantics guided exploration.

Evaluation Setup of RQ3. DmsDroid extracts the DMFs and use the functional semantics to guided the exploration, which aims to find more deep-hidden bugs. Thus, we implemented two versions of DmsDroid as the baselines to evaluate the effectiveness of these strategies.

- DmsDroid$^{\alpha}$: This baseline comparison aims to evaluate the effectiveness of the functional semantics guided exploration. Specifically, for the workflow in Fig. 3, DmsDroid$^{\alpha}$ only runs $Stage_1$ (Depth-first Exploration) but without $Stage_2$ (Functional Semantics Guided Exploration). In the second hour of testing, DmsDroid$^{\alpha}$ will no longer give rewards. If DmsDroid outperforms DmsDroid$^{\alpha}$, functional semantics guided exploration is proven beneficial.
- DmsDroid$^{\beta}$: This baseline comparison aims to evaluate the necessity of extracting DMFs, the effectiveness of the reward function based on DMFs and DMFSQs. Specifically, for the workflow in Fig. 3, DmsDroid$^{\beta}$ only runs $Stage_2$ (Functional semantics guided exploration) and limits the functional reward to only $h(s, a)$. If DmsDroid outperforms DmsDroid$^{\beta}$, it proves the necessity of DMF extraction and the effectiveness of the reward function based on DMFs and DMFSQs.

5.2 Experimental Results

Answer to RQ1: Figure 4 shows the effectiveness of DmsDroid in extracting DMFs. The horizontal axis marks metrics, and vertical axis corresponds to the values. DmsDroid was able to extract 87.72% DMFs (recall values range from 0.577 to 1.0), indicating that the extraction strategy is effective.

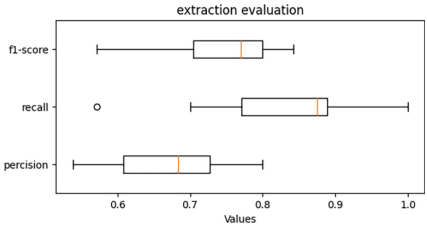


Fig. 4. DMF extraction evaluation Result

However, DmsDroid does not extract DMFs accurately enough (prediction values range from 0.535 to 0.809). We speculate that there are three reasons. First, some developers name DMF-related buttons without clear semantics during the development process. In GrowTracker, the *Navigate up* button submits new plant data. DmsDroid has no way to recognize such a submit button, resulting in missing some DMFs. Second, there are many useless extracted event sequences, most of them belong to the wrongly extracted *Read* DMF, whose pattern is relatively simple. Thirdly, during the automatic extraction of DMF, the explored actions are not explored again, leading to miss some DMFs.

Overall, the result of extraction experiment is reasonable. The recall rate shows that the most of confirmed DMFs can be executed. DmsDroid can do subsequent functional semantics guide exploration based on the extracted DMFs.

In summary, DmsDroid *can extract DMFs in real time within one hour of exploration, and its integrity is guaranteed.*

Answer to RQ2: Table 2 shows the achieved code coverage and revealed bugs of five testing tools (Monkey, Q-testing, ComboDroid, PbfDroid and DmsDroid) and the gray cell denotes which tool covers the highest coverage or discovers the most bugs. The last line in Table 2 gives the overall average result of 16 apps. For coverage, DmsDroid achieves 44.75% on average and much

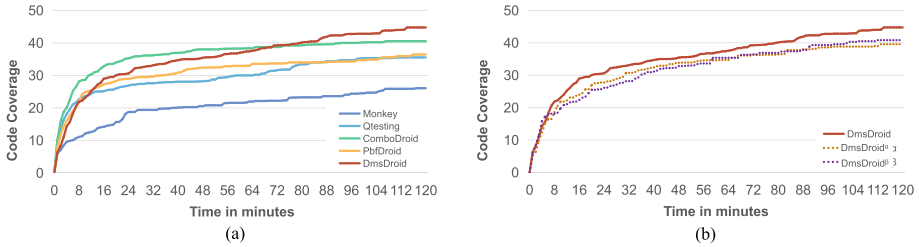


Fig. 5. The achieved code coverage over execution time. (a) shows the coverage changes of DmsDroid compared with other testing tools; (b) shows the coverage changes of DmsDroid compared with its two versions (DmsDroid$^{\alpha}$ and DmsDroid$^{\beta}$).

higher than Monkey (26.06%), Q-testing (35.53%), ComboDroid (40.53%) and PbfDroid(36.43%). For bugs, DmsDroid detects 22 bugs, which is 1.8 times Monkey and Q-testing (12), 1.7 times ComboDroid (13) and 1.46 times PbfDroid(15). Note that the bug detection efficacy of PbfDroid in this paper is limited as the bug definition here excludes functional bugs.

Figure 5.a depict the average instruction coverage over execution time. Monkey has a relatively slower coverage growth due to its random strategy. Before the 13th minute, DmsDroid does not perform as well as others. This situation is because (1) Q-testing is driven by curiosity exploration; (2) PbfDroid uses DMF extracted manually in advance for combination; (3) ComboDroid fully combines a large number of event sequences with dataflow. However, since PbfDroid relies heavily on manual DMF collection, it miss simple operational bugs, resulting in subsequent slow coverage growth. After the 71th minute, the code coverage of DmsDroid gradually exceeds that of ComboDroid. Because in the second stage, DmsDroid adopts the exploration strategy of reinforcement learning, to effectively combine the DMFs generated in $Stage_1$.

To prove the authenticity of the bugs discovered by DmsDroid, we have reported these bugs to developers. So far, 7 reported bugs have been confirmed, shown in Table 3. For the bugs triggered by DMFs, Monkey and Q-testing is hard to detect, because they are more concerned with the single execution event, rather than the complete execution of the functions. ComboDroid, PbfDroid and DmsDroid could detect the most deep-hidden bugs triggered by DMFs. However, ComboDroid and PbfDroid use a cross-enumeration method to combine these sequences, which takes longer to detect deep-hidden problems.

In summary, DmsDroid *achieves the highest code coverage and reveals the most bugs on 16 target apps when comparing to other testing tools.*

Answer to RQ3. Table 2 shows the testing effectiveness of DmsDroid$^{\alpha}$ and DmsDroid$^{\beta}$. For the achieved code coverage, DmsDroid achieve 44.75% code coverage higher than DmsDroid$^{\alpha}$ (39.56%) and DmsDroid$^{\beta}$ (40.81%). For bugs, DmsDroid detects 20 bugs, more than DmsDroid$^{\alpha}$(15) and DmsDroid$^{\beta}$(16).

Table 3. Confirmed bugs by testing tools

ID	App Name	Discovered by	status	Functional semantics	Cause
1	GrowTracker	All	confirmed	Create, Update	IndexoutOfboundExceptions
2	Anymemo	Dm, Cb, Pb	waiting	Create, Read, Delete	NullPointerExceptions
3	K9mail	Dm	fixed	Create	SettingsImportExportException
4	Uhabits	Dm,Q,M	fixed	–	ClassNotFoundException
5	PhonographPlus	Dm, Pb	fixed	Create, Read,Update	IllegalArgumentException
6	Track&graph	All	confirmed	–	ConcurrentModificationException
7	Amaze	Dm	confirmed	Create	StringIndexOutOfBoundsException
8	Amaze	Dm,Q,Cb,Pb	fixed	Create,Delete(Update)	NullPointerException

[1] **All** denotes that all tools dircovered this bug. **M**, **Q**, **Cb**, **Pb** and **Dm** denote Monkey, Q-testing, ComboDroid, PbfDroid and DmsDroid, respectively.

Figure 5.b shows the achieved code coverage over execution time for DmsDroid, DmsDroid$^{\alpha}$ and DmsDroid$^{\beta}$. The coverage changes of DmsDroid is better than DmsDroid$^{\alpha}$ and DmsDroid$^{\beta}$, which proves the usefulness of DMFs. The coverage changes of DmsDroid$^{\beta}$ and DmsDroid$^{\alpha}$ are similar. DmsDroid$^{\alpha}$ use depth-first strategy to fully explore the paths in AUT, while DmsDroid$^{\beta}$ uses the exploration strategy of reinforcement learning to perform more operations related to DMFs. Since DmsDroid$^{\beta}$ will briefly fall into the sparse reward problem of reinforcement learning when not enough events are clicked, the code coverage is lower than DmsDroid$^{\alpha}$ before 59th minutes. After breaking away from sparse rewards, the code coverage of DmsDroid$^{\beta}$ gradually surpassed DmsDroid$^{\alpha}$.

When the 2-hour test ends, the coverage and bug detection effect of DmsDroid$^{\alpha}$ is relatively worse than that of DmsDroid$^{\beta}$. Because DmsDroid$^{\alpha}$ only uses depth-first exploration to traverse the AUT, while DmsDroid$^{\beta}$ uses reinforcement learning to enable the tester to continuously learn to adjust events, and generate test cases with DMFs to improve testing efficiency.

In summary, DmsDroid *can generate meaningful DMFSQs for testing. Both the extraction of DMFs and the use of DMFSQs for exploration are essential.*

6 Related Work

Recently, there has been a surge of interest in the realm of automated testing for Android apps. The work related to the paper is as follows:

Generic Automated Testing Tools for Android Apps. A variety of automated GUI testing tools have been developed for ensuring the reliability and quality of Android apps. Random testing tools [1,8,18,25] utilize stochastic methods to explore the app's behavior and generating random sequences of events. Model-based testing tools [7,10,11,13,24,32] leverage formal models of the app to systematically generate test cases. Search-based testing tools [5,12,22,26,27] efficiently find bugs by searching through lots of possible test inputs.

Reinforcement Learning-based Testing Tools. The reinforcement learning based testing tools [4,19–21,30,31,35] dynamically adapt the test strategy based on feedback from the app's execution, optimizing the testing process over time. Q-testing [30] designs a curiosity-driven exploration strategy to explore previously unvisited GUI pages. ARES [31] and DQT [21] uses deep reinforcement learning for black-box testing of apps. IccDroid [15] designs an ICC-guided exploration strategy to systematically test intents in the app.

Testing Tools Related with Functional Semantics. The testing exploration strategies of some tools [14,28,33] are related to functional semantics. SQDroid [14] designs a dynamic semantic reward function to understand the business logic of apps to find deep bugs. PbfDroid [33] finds DMFs in the app and detects the bugs by randomly interleaving the functions. Augusto [28] tests one CRUD operation independently, and does not consider their combinations.

The testing tools do not explicitly account for DMF execution. Our approach find the app crashes guided by the functional semantics in DMFs. To ensure the efficiency of bug detection, we use Q-learning to guide testing exploration.

7 Conclusion

In this paper, we propose a novel approach to find deep-hidden bugs triggered by data manipulation functions (DMFs). After surveying app GUIs, we defined an extraction rule and designed a real-time algorithm to extract DMFs while detecting regular bugs. To detect deep-hidden bugs, we design the Q-learning based functional semantics guided exploration strategy by extracted DMFs. The realization of our idea, DmsDroid, has detected 1.4 to 1.8 times more bugs in 16 apps, outperforming other tools. We have reported previously unreported bugs to the developers, with 75% of them being related to DMFs. So far, 7 reported bugs have been confirmed and 4 of which have already been fixed. In our future work, we will enhance DmsDroid's performance and explore ways to generate bug reports with functional semantics during testing to assist developers to reproduce deep-hidden bugs.

References

1. Ui/application exerciser monkey (2022). https://developer.android.com/studio/test/monkey
2. Android debug bridge (2023). https://developer.android.com/studio/command-line/adb
3. Logcat command-line tool (2023). https://developer.android.com/studio/command-line/logcat
4. Adamo, D., Khan, M.K., Koppula, S., Bryce, R.: Reinforcement learning for android GUI testing. In: Proceedings of the 9th ACM SIGSOFT International Workshop on Automating TEST Case Design, Selection, and Evaluation, pp. 2–8 (2018)

5. Amalfitano, D., Amatucci, N., Fasolino, A.R., Tramontana, P.: Agrippin: a novel search based testing technique for android applications. In: Proceedings of the 3rd International Workshop on Software Development Lifecycle for Mobile, pp. 5–12 (2015)
6. Amalfitano, D., Fasolino, A.R., Tramontana, P., De Carmine, S., Memon, A.M.: Using GUI ripping for automated testing of android applications. In: Proceedings of the 27th IEEE/ACM International Conference on Automated Software Engineering, pp. 258–261 (2012)
7. Amalfitano, D., Fasolino, A.R., Tramontana, P., Ta, B.D., Memon, A.M.: Mobiguitar: automated model-based testing of mobile apps. IEEE Softw. **32**(5), 53–59 (2015). https://doi.org/10.1109/MS.2014.55
8. Arcuri, A., Briand, L.: A hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering. Softw. Test. Verif. Reliab. **24**(3), 219–250 (2014)
9. Azim, T., Neamtiu, I.: Targeted and depth-first exploration for systematic testing of android apps. In: Proceedings of the 2013 ACM SIGPLAN International Conference on Object Oriented Programming Systems Languages & Applications, pp. 641–660 (2013)
10. Baek, Y.M., Bae, D.H.: Automated model-based android GUI testing using multi-level GUI comparison criteria. In: Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering, pp. 238–249 (2016)
11. Cai, T., Zhang, Z., Yang, P.: Fastbot: a multi-agent model-based test generation system Beijing Bytedance network technology co., ltd. In: Proceedings of the IEEE/ACM 1st International Conference on Automation of Software Test, pp. 93–96 (2020)
12. Dong, Z., Böhme, M., Cojocaru, L., Roychoudhury, A.: Time-travel testing of android apps. In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, pp. 481–492 (2020)
13. Gu, T., et al.: Practical GUI testing of android applications via model abstraction and refinement. In: 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE), pp. 269–280. IEEE (2019)
14. Guo, H., Liu, X., Li, B., Cai, L., Hu, Y., Cao, J.: Sqdroid: a semantic-driven testing for android apps via q-learning. In: 2021 IEEE 21st International Conference on Software Quality, Reliability and Security (QRS), pp. 301–310. IEEE (2021)
15. Guo, H., Su, T., Liu, X., Gu, S., Sun, J.: Effectively finding ICC-related bugs in android apps via reinforcement learning. In: 34th IEEE International Symposium on Software Reliability Engineering (ISSRE) (2023)
16. Hu, C., Neamtiu, I.: Automating GUI testing for android applications. In: Proceedings of the 6th International Workshop on Automation of Software Test, pp. 77–83 (2011)
17. KG, M.G..C., Contributors: Jacoco - java code coverage library (2019). https://www.jacoco.org/jacoco/trunk/index.html. Accessed 7 Aug 2019
18. Kong, P., Li, L., Gao, J., Liu, K., Bissyandé, T.F., Klein, J.: Automated testing of android apps: a systematic literature review. IEEE Trans. Reliab. **68**(1), 45–66 (2018)
19. Koroglu, Y., Sen, A.: Reinforcement learning-driven test generation for android GUI applications using formal specifications. arXiv preprint arXiv:1911.05403 (2019)
20. Koroglu, Y., et al.: QBE: qlearning-based exploration of android applications. In: 2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST), pp. 105–115. IEEE (2018)

21. Lan, Y., et al.: Deeply reinforcing android GUI testing with deep reinforcement learning. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, pp. 1–13 (2024)
22. Li, Y., Yang, Z., Guo, Y., Chen, X.: Droidbot: a lightweight UI-guided test input generator for android. In: 2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C), pp. 23–26. IEEE (2017)
23. Li, Y., Yang, Z., Guo, Y., Chen, X.: Humanoid: a deep learning-based approach to automated black-box android app testing. In: 2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 1070–1073. IEEE (2019)
24. Lv, Z., Peng, C., Zhang, Z., Su, T., Liu, K., Yang, P.: Fastbot2: reusable automated model-based GUI testing for android enhanced by reinforcement learning. In: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, pp. 1–5 (2022)
25. Machiry, A., Tahiliani, R., Naik, M.: Dynodroid: an input generation system for android apps. In: Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering, pp. 224–234 (2013)
26. Mahmood, R., Mirzaei, N., Malek, S.: Evodroid: segmented evolutionary testing of android apps. In: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, pp. 599–609 (2014)
27. Mao, K., Harman, M., Jia, Y.: Sapienz: multi-objective automated testing for android applications. In: Proceedings of the 25th International Symposium on Software Testing and Analysis, pp. 94–105 (2016)
28. Mariani, L., Pezzè, M., Zuddas, D.: Augusto: exploiting popular functionalities for the generation of semantic GUI tests with oracles. In: Proceedings of the 40th International Conference on Software Engineering, pp. 280–290 (2018)
29. Nguyen, C.D., Marchetto, A., Tonella, P.: Combining model-based and combinatorial testing for effective test case generation. In: Proceedings of the 2012 International Symposium on Software Testing and Analysis, pp. 100–110 (2012)
30. Pan, M., Huang, A., Wang, G., Zhang, T., Li, X.: Reinforcement learning based curiosity-driven testing of android applications. In: Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 153–164 (2020)
31. Romdhana, A., Merlo, A., Ceccato, M., Tonella, P.: Deep reinforcement learning for black-box testing of android apps. ACM Trans. Softw. Eng. Methodol. (TOSEM) **31**(4), 1–29 (2022)
32. Su, T., et al.: Guided, stochastic model-based GUI testing of android apps. In: Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering, pp. 245–256 (2017)
33. Sun, J., Su, T., Jiang, J., Wang, J., Pu, G., Su, Z.: Property-based fuzzing for finding data manipulation errors in android apps. In: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '23). ACM (2023)
34. Sutton, R.S., Barto, A.G.: Reinforcement Learning: An Introduction. MIT Press, Cambridge (2018)
35. Vuong, T.A.T., Takada, S.: A reinforcement learning based approach to automated testing of android applications. In: Proceedings of the 9th ACM SIGSOFT International Workshop on Automating TEST Case Design, Selection, and Evaluation, pp. 31–37 (2018)

36. Wang, J., Jiang, Y., Xu, C., Cao, C., Ma, X., Lu, J.: Combodroid: generating high-quality test inputs for android apps via use case combinations. In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, pp. 469–480 (2020)
37. Watkins, C.J., Dayan, P.: Q-learning. Mach. Learn. **8**, 279–292 (1992)
38. Zhang, J., Sagar, S., Shihab, E.: The evolution of mobile apps: an exploratory study. In: Proceedings of the 2013 International Workshop on Software Development Lifecycle for Mobile, pp. 1–8 (2013)